

# Neural Networks + Learning

## Questions & Model Answers + Canonical Worked Numerical

Past Paper Questions & Solutions (2020–2024)  
CSE 713 Artificial Intelligence

Study Notes — June 2026

### Contents

---

|                                                       |    |
|-------------------------------------------------------|----|
| Exam Pattern Summary                                  | 2  |
| 1 2024 — Q8 (9 marks)                                 | 3  |
| 2 2023 — Q8 (9 marks)                                 | 5  |
| 3 2022 — B-1 (9 marks)                                | 7  |
| Canonical Worked Numerical (Han & Kamber Example 9.1) | 9  |
| Quick Reference Summary                               | 11 |

## Exam Pattern Summary

### Key Exam Facts

- **Yield: 3 of the 5 years (2024, 2023, 2022)** ask a Neural Networks question — *not every year*, despite what the old map said. 9 marks each time it appears, always Section B.
- **Always asked:** ANN definition + comparison with biological networks (2024, 2022) or “associative memory / multilayer architecture” framing (2023).
- **Sometimes asked (1/5 years so far — 2023 only):** a numerical — sigmoid forward-pass / backward-pass / weight-update calculation. This is the highest-value thing to drill, because it is exactly the style of the canonical Han & Kamber Example 9.1 (reproduced in full below).
- **Always bundled:** Fuzzy Logic + Uncertainty short notes appear as the closing sub-part in 2024 and 2022 (see `FuzzyLogicSolutions.tex`).
- **Strategy:** memorise (a) the ANN-vs-biological comparison cold, (b) the associative-memory + multilayer-architecture paragraph, and (c) walk through the Example 9.1 numerical by hand at least 3 times — if a numerical appears again, it will look just like it.

### Map Correction Note (2026-06-07)

The original `_TopicQuestionMap.md` claimed Neural Networks appears in **5/5 years**, listing “2021 — Q8 (9 marks) — NN + Burglary Alarm Bayesian Network”. Both the 2021 and 2020 raw papers were re-read page by page to verify this:

- **2021 Q8** is *entirely* a Bayesian Network question — two independent alarm sensors, Burglary  $\rightarrow$  Alarm1/Alarm2, full CPTs for Burglary/Earthquake/Alarm/JohnCalls/MaryCalls, joint-distribution computation. **Zero Neural Network content.**
- **2020** contains no Neural Network question anywhere in the paper (Group A Q1–4, Group B Q5–8); that year’s Q8 is Bayesian Network + meningitis (a Bayes’ Theorem numerical), not NN.

**Corrected true yield: 3/5 years (2024, 2023, 2022 only).** The map, the Quick Lookup Table, and the CSP topic row (which had the same false “combined with NN” claim attached to 2020) have all been corrected. This still makes NN a high-value topic — 9 marks, 3 years out of 5 — just not a guaranteed-every-year one.

## 2024 — Q8 (9 marks)

## 2024 Section B, Q8

- (a) What is learning? Describe the inductive learning method with an example. (3)
- (b) Compare artificial and biological neural networks. What aspects of biological networks are not mimicked by artificial ones? What aspects are similar? (3)
- (c) Write short notes on: i) Fuzzy logic ii) Uncertainty (3) — see *FuzzyLogicSolutions.tex* for part (c)

## Solution (a) — Learning &amp; Inductive Learning

**What is learning?** Learning is the process by which an [redacted] on a task by [redacted] or [redacted] based on [redacted] — in ANNs specifically, learning means [redacted] so the network's [redacted] approaches a [redacted]

**Inductive learning:** a form of [redacted] in which the learner is [redacted] a [redacted] (the [redacted] and must [redacted] that [redacted] which then [redacted]. It moves from [redacted] to a [redacted] (the opposite of deduction).

**Worked example:** Given examples of the [redacted] as pairs (1, 1), (2, 4), (3, 9), (4, 16), an inductive learner hypothesises the general rule  $f(x) = x^2$  and can now predict  $f(5) = 25$  without being told. In an ANN context: shown many (image, label) pairs of handwritten digits, the network induces a general weight-pattern that classifies digits it has never seen.

**Key property:** inductive learning is inherently *uncertain* — multiple hypotheses can fit the training data (the problem of induction / Hume's problem), so the learner must use a bias (e.g., prefer the simplest consistent hypothesis — Occam's razor) to choose among them.

## Solution (b) — Artificial vs Biological Neural Networks

Structural mapping (memorise this table):

| Biological | Artificial              |
|------------|-------------------------|
| [redacted] | [redacted]              |
| [redacted] | [redacted]              |
| [redacted] | [redacted]              |
| [redacted] | [redacted]              |
| [redacted] | Fast speed ( $\sim$ ns) |
| [redacted] | [redacted]              |

Aspects *NOT* mimicked by artificial networks:

- True massive parallelism and scale ( $10^{11}$  neurons,  $10^{14}$  synapses vs. a few thousand–million units)
- Chemical (neurotransmitter-based) signalling — ANNs use only numeric weighted sums
- Continuous, asynchronous, self-organising growth and rewiring (neuroplasticity) during the system’s lifetime
- Energy efficiency — the brain runs on  $\sim 20\text{W}$ ; large ANNs require kilowatts of GPU power
- Genuine understanding / consciousness / embodied sensorimotor grounding

**Aspects that *are* similar:**

- Both are networks of simple processing units connected by weighted links
- Both *learn* by adjusting connection strengths (synaptic strength  $\leftrightarrow$  weight) based on experience
- Both produce an output as some (thresholded/activation) function of a weighted sum of inputs
- Both exhibit *distributed representation* — knowledge is spread across many connections, not stored in one place (hence the “black box” property)
- Both can generalise from examples to novel situations

## 2023 — Q8 (9 marks)

## 2023 Section B, Q8

- (a) What is associative memory? Is it your belief that neural networks are fundamentally rooted in associative memory? Explain considering the structure and principles of multilayer neural network architecture. (3)
- (b) Suppose neurons in a network use the sigmoid activation function and the network performs a forward pass followed by a backward pass to update its weights. The true output is  $y = 0.6$  and the learning rate is 0.8. The network has weights  $W_{13} = 0.1$ ,  $W_{14} = 0.4$ ,  $W_{23} = 0.8$ ,  $W_{24} = 0.6$ ,  $W_{35} = 0.3$ ,  $W_{45} = 0.9$  and inputs  $x_1 = 0.35$ ,  $x_2 = 0.9$ . Execute an additional forward pass with the updated network. (6)

## Solution (a) — Associative Memory &amp; Multilayer Architecture

**Definition:** Associative memory is a memory organisation principle in which stored information (a pattern) is recalled not by its address/location but by *content* — presenting a partial, noisy, or distorted version of a pattern is enough to retrieve the complete, correct stored pattern (“pattern completion”). Hopfield networks are the canonical ANN realisation of this idea.

**Are neural networks fundamentally rooted in associative memory? — Yes, and here is why:**

- A multilayer (feedforward) network learns by storing an input–output *association* ( $\mathbf{x} \rightarrow \mathbf{y}$ ) distributed across its weight matrix — exactly the content-addressable principle of associative memory, just generalised from binary patterns to continuous function approximation.
- **Structure:** an input layer receives the pattern, one or more hidden layers build an internal (distributed) representation of it, and the output layer produces the associated response. Every unit’s output becomes the next layer’s input via weighted, summed, activated connections (typically sigmoid).
- **Principle:** during training (e.g. via backpropagation), the network strengthens the weight pathways that connect a given input pattern to its correct output — precisely the “store an association by adjusting connection strength” mechanism that defines associative memory, just realised through gradient descent rather than a one-shot Hebbian rule.
- **Generalisation:** because the association is stored in *distributed* weights rather than at discrete addresses, the network can recall/produce correct (or close) outputs even for inputs it has not seen before — the same robustness-to-partial-input that defines associative recall.

**Conclusion (one-line for the exam):** Yes — both single-layer associative memories (e.g. Hopfield nets) and multilayer feedforward networks store knowledge as distributed weighted associations between inputs and outputs, and recall/generalise via the same content-based, partial-pattern-completion principle; multilayer architectures simply extend this to richer, hierarchical, non-linear associations.

**Solution (b) — Sigmoid Forward + Backward Pass Numerical**

**Step 0 — Network layout (from the given weight subscripts):** Units 1, 2 = inputs; units 3, 4 = hidden layer; unit 5 = output. Connections: 1→3 ( $W_{13}=0.1$ ), 1→4 ( $W_{14}=0.4$ ), 2→3 ( $W_{23}=0.8$ ), 2→4 ( $W_{24}=0.6$ ), 3→5 ( $W_{35}=0.3$ ), 4→5 ( $W_{45}=0.9$ ).

Sigmoid:  $\sigma(z) = \frac{1}{1 + e^{-z}}$ , with derivative  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ . (Assume biases = 0 unless given, as in the canonical Example 9.1 layout.)

**Step 1 — Forward pass (compute hidden & output activations):**

$$net_3 = W_{13}x_1 + W_{23}x_2 = (0.1)(0.35) + (0.8)(0.9) = 0.035 + 0.72 = 0.755$$

$$O_3 = \sigma(0.755) = \frac{1}{1 + e^{-0.755}} \approx 0.6803$$

$$net_4 = W_{14}x_1 + W_{24}x_2 = (0.4)(0.35) + (0.6)(0.9) = 0.14 + 0.54 = 0.68$$

$$O_4 = \sigma(0.68) = \frac{1}{1 + e^{-0.68}} \approx 0.6637$$

$$net_5 = W_{35}O_3 + W_{45}O_4 = (0.3)(0.6803) + (0.9)(0.6637) \approx 0.2041 + 0.5973 = 0.8014$$

$$O_5 = \sigma(0.8014) \approx 0.6902$$

**Step 2 — Backward pass (error & deltas, using  $T = y = 0.6$ ):**

$$\begin{aligned} Err_5 &= O_5(1 - O_5)(T - O_5) = (0.6902)(0.3098)(0.6 - 0.6902) \\ &= (0.2138)(-0.0902) \approx -0.01928 \end{aligned}$$

$$Err_3 = O_3(1 - O_3) Err_5 W_{35} = (0.6803)(0.3197)(-0.01928)(0.3) \approx -0.00126$$

$$Err_4 = O_4(1 - O_4) Err_5 W_{45} = (0.6637)(0.3363)(-0.01928)(0.9) \approx -0.00387$$

**Step 3 — Weight updates ( $\Delta w_{ij} = l \cdot Err_j \cdot O_i$ ,  $l = 0.8$ ):**

$$\Delta W_{35} = (0.8)(-0.01928)(0.6803) \approx -0.01049 \Rightarrow W_{35}^{new} \approx 0.3 - 0.0105 = 0.2895$$

$$\Delta W_{45} = (0.8)(-0.01928)(0.6637) \approx -0.01024 \Rightarrow W_{45}^{new} \approx 0.9 - 0.0102 = 0.8898$$

$$\Delta W_{13} = (0.8)(-0.00126)(0.35) \approx -0.000353 \Rightarrow W_{13}^{new} \approx 0.0996$$

$$\Delta W_{23} = (0.8)(-0.00126)(0.9) \approx -0.000907 \Rightarrow W_{23}^{new} \approx 0.7991$$

$$\Delta W_{14} = (0.8)(-0.00387)(0.35) \approx -0.001084 \Rightarrow W_{14}^{new} \approx 0.3989$$

$$\Delta W_{24} = (0.8)(-0.00387)(0.9) \approx -0.002789 \Rightarrow W_{24}^{new} \approx 0.5972$$

**Step 4 — The actual question: re-run the forward pass with updated weights:**

$$net'_3 = (0.0996)(0.35) + (0.7991)(0.9) \approx 0.0349 + 0.7192 = 0.7541 \Rightarrow O'_3 = \sigma(0.7541) \approx 0.6802$$

$$net'_4 = (0.3989)(0.35) + (0.5972)(0.9) \approx 0.1396 + 0.5375 = 0.6771 \Rightarrow O'_4 = \sigma(0.6771) \approx 0.6631$$

$$net'_5 = (0.2895)(0.6802) + (0.8898)(0.6631) \approx 0.1969 + 0.5901 = 0.7870 \Rightarrow O'_5 = \sigma(0.7870) \approx 0.687$$

**Result:** the new network output is  $O'_5 \approx 0.6871$  — moved *closer* to the true output  $y = 0.6$  than the original  $O_5 \approx 0.6902$  (error shrank from  $-0.0902$  to  $-0.0871$ ), demonstrating the weight update is working in the correct direction. (In the exam, showing this directional improvement explicitly is what earns the final mark — exact decimals matter less than the correct procedure and the correct conclusion.)

## 2022 — B-1 (9 marks)

## 2022 Section B, B-1

- (a) What is an ANN? Compare artificial and biological networks — which aspects of biological networks are not mimicked by artificial ones, and which are similar? (3)
- (b) What is learning? Describe the inductive learning method with an example. (2)
- (c) Explain supervised, unsupervised, and reinforcement learning. (2)
- (d) Write short notes on: i) Fuzzy logic ii) Uncertainty (2) — see *FuzzyLogicSolutions.tex* for part (d)

## Solution (a) — What is an ANN? + Comparison

**Definition:** An Artificial Neural Network (ANN) is a computational model loosely inspired by the structure of biological brains — a network of simple processing units (*nodes/neurons*), connected by weighted links, that computes its output as a (typically non-linear, e.g. sigmoid) function of the weighted sum of its inputs, and *learns* by adjusting those weights from examples.

**Comparison:** *identical content to 2024 Q8b above* — reuse the soma/dendrite/axon/synapse  $\leftrightarrow$  node/input/output/weight mapping table, the “not mimicked” list (scale, chemical signalling, neuroplasticity, energy efficiency, consciousness) and the “similar” list (weighted-sum + threshold computation, learning by adjusting connection strength, distributed representation, generalisation). With only 3 marks here, give the definition + 2 “not mimicked” + 2 “similar” points.

## Solution (b) — Learning + Inductive Learning (condensed)

Identical core content to 2024 Q8a, trimmed to 2 marks: *Learning* = improving performance via experience (in ANNs: adjusting weights toward a target). *Inductive learning* = inferring a general rule from specific input–output examples that generalises to unseen cases (e.g. learning  $f(x) = x^2$  from the pairs (1, 1), (2, 4), (3, 9) and predicting  $f(4) = 16$ ).

## Solution (c) — Supervised vs Unsupervised vs Reinforcement Learning

- **Supervised learning:** the learner is given labelled training examples — pairs of (input, correct output/target) — and adjusts its parameters to minimise the error between its prediction and the target. *Example:* backpropagation training a network to classify loan applications as “approve”/“reject” given historical labelled cases.
- **Unsupervised learning:** the learner is given only inputs, with *no* target outputs, and must discover structure, patterns, clusters, or representations on its own. *Examples:* Hopfield networks (pattern/cluster storage &

recall), Adaptive Resonance Theory (ART), Self-Organising Feature Maps (SOFM/Kohonen maps).

- **Reinforcement learning:** the learner (agent) interacts with an environment, takes actions, and receives delayed *reward/punishment* signals (not direct correct-answer labels); it learns a policy that maximises cumulative reward through trial-and-error exploration. *Example:* an agent learning to play a game by being rewarded for winning moves and penalised for losing ones, without ever being told the “correct” move directly.

**One-line distinguishing test (useful if marks are tight):** supervised = “here is the right answer for each example”; unsupervised = “here is data, find the structure yourself”; reinforcement = “here is a reward signal after you act — learn from the consequences”.



## Canonical Worked Numerical — Han & Kamber Example 9.1

### Why memorise this specific example

This is the *exact textbook example* (back propagation.pdf, Han & Kamber *Data Mining* Ch. 9.2, Figure 9.5 / Tables 9.1–9.4) that the 2023 Q8b numerical is modelled on — same network shape (3 input  $\rightarrow$  2 hidden  $\rightarrow$  1 output), same sigmoid activation, same forward  $\rightarrow$  error  $\rightarrow$  weight-update structure. If a numerical appears on your exam, walking through *this* derivation by hand 2–3 times is the highest-value 20 minutes you can spend on this topic.

### Setup

Network: 3 input units (1, 2, 3)  $\rightarrow$  2 hidden units (4, 5)  $\rightarrow$  1 output unit (6), fully connected, biases  $\theta_4, \theta_5, \theta_6$  included. Input tuple  $X = (x_1, x_2, x_3) = (1, 0, 1)$ , true class label  $T_6 = 1$ , learning rate  $l = 0.9$ . Initial weights/biases are given in Table 9.2.

**Step 1 — Forward propagation (Table 9.3):**  $I_j = \sum_i w_{ij}O_i + \theta_j$ ,  $O_j = \frac{1}{1 + e^{-I_j}}$

| Unit $j$ | Net input $I_j$  | /                                 | Output $O_j$ |
|----------|------------------|-----------------------------------|--------------|
| 4        | $I_4 = -0.7$ ,   | $O_4 = 1/(1 + e^{0.7}) = 0.332$   |              |
| 5        | $I_5 = 0.1$ ,    | $O_5 = 1/(1 + e^{-0.1}) = 0.525$  |              |
| 6        | $I_6 = -0.105$ , | $O_6 = 1/(1 + e^{0.105}) = 0.474$ |              |

**Step 2 — Backpropagating the error (Table 9.3 cont.):**  $Err_j = O_j(1 - O_j)(T_j - O_j)$  for o

$$Err_6 = O_6(1 - O_6)(T_6 - O_6) = (0.474)(1 - 0.474)(1 - 0.474) = 0.1311$$

$$Err_5 = O_5(1 - O_5) Err_6 w_{56} = (0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$$

$$Err_4 = O_4(1 - O_4) Err_6 w_{46} = (0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$$

**Step 3 — Updating weights and biases (Table 9.4):**  $\Delta w_{ij} = l \cdot \text{Err}_j \cdot O_i$ ,  $w_{ij}^{\text{new}} = w_{ij} + \Delta w_{ij}$

$$w_{46} = -0.3 + (0.9)(0.1311)(0.332) = -0.261$$

$$w_{56} = -0.2 + (0.9)(0.1311)(0.525) = -0.138$$

$$w_{14} = 0.2 + (0.9)(-0.0087)(1) = 0.192$$

$$w_{15} = -0.3 + (0.9)(-0.0065)(1) = -0.306$$

$$w_{24} = 0.4 + (0.9)(-0.0087)(0) = 0.4 \quad (\text{unchanged} — O_2 = x_2 = 0)$$

$$w_{25} = 0.1 + (0.9)(-0.0065)(0) = 0.1 \quad (\text{unchanged})$$

$$w_{34} = -0.5 + (0.9)(-0.0087)(1) = -0.508$$

$$w_{35} = 0.2 + (0.9)(-0.0065)(1) = 0.194$$

$$\theta_6 = 0.1 + (0.9)(0.1311) = 0.218, \quad \theta_5 = 0.2 + (0.9)(-0.0065) = 0.194, \quad \theta_4 = -0.4 + (0.9)(-0.0087) = -0.408$$

**The pattern to internalise (applies to ANY similar numerical, including 2023 Q8b)**

1. **Forward:** net input = weighted sum of inputs + bias, then squash with sigmoid  $\rightarrow$  unit output.
2. **Error (output unit):**  $\text{Err} = O(1 - O)(T - O)$  — “derivative of sigmoid”  $\times$  “how wrong we are”.
3. **Error (hidden unit):**  $\text{Err} = O(1 - O) \sum (\text{downstream } \text{Err} \times \text{connecting weight})$  — the error is *passed backward* weighted by the same connections it travelled forward on. This is the literal meaning of “back-propagation”.
4. **Update:**  $\Delta w = l \times \text{Err}_{(\text{receiving unit})} \times O_{(\text{sending unit})}$ , and  $w^{\text{new}} = w^{\text{old}} + \Delta w$ . Biases use the same rule with the “sending output” implicitly = 1.
5. **Sanity check your answer** the way Table 9.4/the 2023 solution does: plug the new weights back through one more forward pass and confirm the output has moved *closer* to the target — this is often an explicit final sub-question (exactly what 2023 Q8b asks for).

## Quick Reference Summary

### Year-by-Year Appearance (2020–2024) — corrected

| Year | Location                 | What it asked                                                       |
|------|--------------------------|---------------------------------------------------------------------|
| 2024 | Section B, Q8 (9 marks)  | Inductive learning + ANN-vs-biological comparison + Fuzzy/U         |
| 2023 | Section B, Q8 (9 marks)  | Associative memory/multilayer architecture + <b>sigmoid forward</b> |
| 2022 | Section B, B-1 (9 marks) | ANN definition+comparison + learning + supervised/unsuperv          |
| 2021 | —                        | <b>Not asked</b> (Q8 that year is a pure Bayesian Network question) |
| 2020 | —                        | <b>Not asked</b> (no NN question anywhere in the paper — correcte   |

### Five Formulas to Have Cold

1. Sigmoid:  $\sigma(z) = \frac{1}{1 + e^{-z}}$ , derivative  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$  — the single most-reused identity in any NN numerical.
2. Net input:  $I_j = \sum_i w_{ij} O_i + \theta_j$  (weighted sum + bias).
3. Output-unit error:  $Err_j = O_j(1 - O_j)(T_j - O_j)$ .
4. Hidden-unit error (backpropagated):  $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$ .
5. Weight/bias update:  $\Delta w_{ij} = l \cdot Err_j \cdot O_i$ ,  $w_{ij}^{new} = w_{ij}^{old} + \Delta w_{ij}$  (and  $\Delta \theta_j = l \cdot Err_j$ ).

### The Two Paragraphs to Memorise Cold

**ANN vs Biological (for the comparison question):** “An ANN is a network of simple processing units connected by weighted links that computes outputs as a function of weighted input sums and learns by adjusting those weights — directly analogous to biological neurons (soma=node, dendrite=input, axon=output, synapse=weight) learning by adjusting synaptic strength. What is *not* mimicked: the brain’s massive parallel scale ( $10^{11}$  neurons), chemical signalling, continuous self-organising neuroplasticity, and extreme energy efficiency. What *is* similar: both are networks of simple units, both learn from experience by changing connection strengths, both compute via weighted-sum-plus-activation, and both store knowledge in a distributed, generalisable, ‘black box’ fashion.”

**Associative memory & multilayer NNs (for the 2023-style question):** “Associative memory recalls a complete stored pattern from a partial or noisy cue, by content rather than address. Multilayer neural networks are fundamentally rooted in this idea: they store an input-output association distributedly across their weight matrix (built up, layer by layer, via forward propagation and activation), and backpropagation is simply the mechanism by which that association is strengthened/corrected from examples — making the network’s learned behaviour a generalised, continuous-valued form of content-addressable, pattern-completing associative recall.”

### Common Mistakes

- Don’t write a generic “ANN = computer brain” definition — name the structural mapping (soma/dendrite/axon/synapse  $\leftrightarrow$  node/input/output/weight)

for marks.

- In the backprop numerical: compute the *output*-unit error first, then propagate *backward* to hidden units — a common slip is computing hidden-unit errors before the output error exists.
- Remember *Err* for a hidden unit uses the *already-updated-direction* weights connecting it *forward* to the next layer (not the weights feeding into it) — this is the crux of “back”-propagation.
- If  $x_i = 0$  for some input unit,  $\Delta w$  for weights leaving that unit is exactly 0 — don’t recompute it as if it were non-zero (a trap many students fall into, mirrored in the canonical example where  $w_{24}, w_{25}$  stay unchanged because  $O_2 = x_2 = 0$ ).
- Don’t confuse *supervised* (has labelled targets), *unsupervised* (no targets, finds structure), and *reinforcement* (delayed reward signal, not direct labels) — give a concrete example for each.
- This topic is *not* guaranteed every year (3/5) — don’t let it crowd out Bayes/BN, FOL, or Planning, all of which are 5/5.